

LAPORAN PRAKTIKUM
JOB SHEET 04 - RELASI KELAS (AGGREGATION, COMPOSITION, DAN
DEPENDENCY)



Disusun Oleh:

Nama : Mohammad Hafidz Rafi' Rabbani
NIM : 254107020084
Kelas : 2H
No. Absen : 14

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan	3
2. Praktikum	3
2.1. Percobaan 1: Aggregation Satu-ke-Satu (Laptop dan Processor)	3
2.1.1. Pertanyaan	5
2.2. Percobaan 2: Aggregation Ganda (Pelanggan, Mobil, dan Sopir)	7
2.2.1. Pertanyaan	8
2.3. Percobaan 3: Aggregation Multi-Role dan Penanganan NullPointerException (Kereta Api dan Pegawai)	10
2.3.1. Pertanyaan	11
2.4. Percobaan 4: Composition Array of Object dan Multiplicity (Gerbong, Kursi, Penumpang)	13
2.4.1. Pertanyaan	15
2.5. Percobaan 5: Composition Kepemilikan Eksklusif (Mobil dan Mesin)	17
2.5.1. Pertanyaan	18
2.6. Percobaan 6: Dependency / Uses-A (Laptop dan Printer)	19
2.6.1. Pertanyaan	20
3. Tugas	22
3.1. Tugas Mandiri 1: Studi Kasus Sistem RPG Battle Berorientasi Objek	22
3.2. Tugas Mandiri 2: Pedoman Keputusan Desain Relasi Antar-Kelas	24

1. Tujuan

- Menjelaskan perbedaan konsep relasi Association, Aggregation, Composition, dan Dependency (uses-a).
- Mengimplementasikan relasi Aggregation antar-kelas untuk kasus satu-ke-satu, relasi ganda, dan relasi multi-role.
- Mengimplementasikan relasi Composition tunggal dan multi-objek (array of objects) serta memahami aturan lifecycle dan pembatasan setter.
- Mengimplementasikan relasi Dependency (uses-a) di mana objek hanya digunakan temporer lewat parameter method tanpa disimpan sebagai atribut.
- Menerapkan guard clause untuk mendeteksi dan mencegah NullPointerException pada referensi objek yang belum diinisialisasi.
- Merancang dan membangun studi kasus mandiri yang mengintegrasikan relasi Aggregation, Composition, dan Dependency dalam satu sistem berorientasi objek.

2. Praktikum

2.1. Percobaan 1: Aggregation Satu-ke-Satu (Laptop dan Processor)

Pada percobaan pertama, dipelajari relasi kepemilikan independen (**has-a**) antara class `Laptop` (Whole) dan class `Processor` (Part). Objek `Processor` diinstansiasi secara mandiri di luar class `Laptop` lalu disuntikkan (**injected**) melalui constructor berparameter atau method setter `setProc()`. Class `Laptop` mendelegasikan pencetakan informasi prosesor melalui pemanggilan method `proc.info()`.

```
// Laptop.java
public class Laptop {
    private String merk;
    private Processor proc;

    public Laptop(String merk, Processor proc) {
        this.merk = merk;
        this.proc = proc;
    }

    public void setProc(Processor proc) {
        this.proc = proc;
    }

    public void info() {
        System.out.println("Merk Laptop = " + merk);
    }
}
```

```
    proc.info();  
}  
}
```

```
● rafi@santaro experiments % java ./MainPercobaan1.java  
Merk Laptop = Thinkpad  
Merk Processor = Intel i5  
Cache Memory = 3,00  
Merk Laptop = Thinkpad  
Merk Processor = Intel i5  
Cache Memory = 4,00  
Merk Laptop = Thinkpad  
Merk Processor = Intel i5  
Cache Memory = 3,00  
○ rafi@santaro experiments %
```

2.1.1. Pertanyaan

1. Di dalam class Processor dan class Laptop, terdapat method setter dan getter untuk masing-masing atributnya. Apakah gunanya method setter dan getter tersebut?
 - Method getter digunakan untuk membaca atau mengambil nilai atribut `private` dari luar kelas (**read-only access**), sedangkan method setter digunakan untuk mengisi, memvalidasi, atau mengubah nilai atribut tersebut secara terkontrol (**write access**) sesuai prinsip enkapsulasi data.
2. Di dalam class Processor dan class Laptop, masing-masing terdapat konstruktor default dan konstruktor berparameter. Bagaimanakah beda penggunaan dari kedua jenis konstruktor tersebut?
 - Konstruktor default digunakan saat ingin membuat objek kosong tanpa memberikan nilai awal langsung pada atributnya (atribut akan bernilai default Java sebelum diatur lewat setter). Sedangkan konstruktor berparameter digunakan untuk langsung memberikan nilai inisialisasi awal pada atribut objek melalui argumen parameter saat objek pertama kali di-instansiasi menggunakan `new`.
3. Perhatikan class Laptop, di antara 2 atribut yang dimiliki (merk dan proc), atribut manakah yang bertipe object? Baris kode manakah yang menunjukkan bahwa class Laptop memiliki relasi dengan class Processor?
 - Atribut yang bertipe object adalah `proc` (bertipe kelas `Processor`). Baris kode yang menunjukkan relasi adalah deklarasi `private Processor proc;` serta parameter constructor `public Laptop(String merk, Processor proc) { this.proc = proc; }` dan setter `public void setProc(Processor proc) { this.proc = proc; }.`
4. Perhatikan pada class Laptop, apakah guna dari sintaks `proc.info()`?
 - Guna sintaks tersebut adalah memanggil method `info()` milik objek `Processor` yang tersimpan pada atribut `proc`. Ini merupakan penerapan pola delegasi: class `Laptop` tidak perlu mencetak rincian teknis prosesor secara manual, melainkan menyuruh objek `Processor` untuk mencetak spesifikasi dirinya sendiri.

5. Pada Langkah 8, objek `p` dibuat lebih dulu baru diberikan ke constructor `Laptop`. Pada Langkah 10, objek `Processor` dibuat langsung di dalam argumen constructor `Laptop` (tanpa variabel `p`). Apakah keduanya menghasilkan output yang berbeda? Mengapa?
- Output yang dihasilkan sama persis. Secara teknis di memori Java (**heap memory**), kedua cara tersebut sama-sama mengalokasikan satu objek `Processor` baru dan mengirimkan alamat referensinya ke constructor `Laptop`. Bedanya, pada Langkah 8 alamat referensi sempat ditampung terlebih dahulu di variabel lokal `p` (`Processor p = new Processor(...)`), sedangkan pada Langkah 10 objek dibuat langsung sebagai **anonymous object** di dalam argumen constructor.
6. Secara kode, apakah relasi `Laptop-Processor` pada percobaan ini termasuk Aggregation atau Composition? Tunjukkan baris kode yang menjadi bukti jawabanmu.
- Termasuk **Aggregation**. Buktinya objek `Processor` dibuat di luar kelas `Laptop` (di `MainPercobaan1`), lalu disuntikkan via constructor atau setter (`public Laptop(String merk, Processor proc)` dan `public void setProc(Processor proc)`). Objek `Processor` memiliki **lifecycle** mandiri dan tidak akan musnah di heap memory jika objek `Laptop` dihapus.
7. Andaikan constructor `Laptop` diubah sehingga `Processor` dibuat sendiri di dalam `Laptop` (`this.proc = new Processor("Generic", 1)`), apakah relasi ini masih Aggregation? Jelaskan alasannya.
- Tidak, relasi tersebut berubah menjadi **Composition**. Hal ini karena objek `Processor` diinstansiasi secara internal menggunakan `new` di dalam constructor `Laptop`, sehingga siklus hidup `Processor` terikat mutlak (**tight coupling**) dengan `Laptop`. Objek `Processor` tidak bisa disuntikkan dari luar dan otomatis musnah jika objek `Laptop` dihapus.

2.2. Percobaan 2: Aggregation Ganda (Pelanggan, Mobil, dan Sopir)

Percobaan kedua menguji skenario di mana satu class Whole (`Pelanggan`) memiliki relasi Aggregation dengan dua class Part yang berbeda (`Mobil` dan `Sopir`). Durasi sewa (`hari`) yang merupakan atribut milik `Pelanggan` dilewatkan sebagai argumen parameter ke method `hitungBiayaMobil(hari)` dan `hitungBiayaSopir(hari)`. Hasil perhitungan kedua objek tersebut kemudian dijumlahkan di method `hitungBiayaTotal()`.

```
// Pelanggan.java
public int hitungBiayaTotal() {
    return mobil.hitungBiayaMobil(hari) + sopir.hitungBiayaSopir(hari);
}
```

```
● rafi@santaro experiments % java ./MainPercobaan2.java
Total Biaya = 1100000
Avanza
○ rafi@santaro experiments %
```

2.2.1. Pertanyaan

1. Perhatikan class Pelanggan. Pada baris program manakah yang menunjukkan bahwa class Pelanggan memiliki relasi dengan class Mobil dan class Sopir?
 - Ditunjukkan pada deklarasi atribut `private Mobil mobil;` dan `private Sopir sopir;`, serta method setter dan getter terkait (`setMobil`, `setSopir`, `getMobil`, `getSopir`).
2. Perhatikan method `hitungBiayaSopir` pada class Sopir, serta method `hitungBiayaMobil` pada class Mobil. Mengapa method tersebut harus memiliki argument hari, padahal hari sendiri adalah atribut milik Pelanggan?
 - Karena tarif sewa mobil dan tarif jasa sopir dihitung berdasarkan durasi harian, tetapi data durasi sewa (`hari`) adalah atribut konteks transaksi yang dimiliki oleh `Pelanggan`. Agar class `Mobil` dan `Sopir` tetap modular dan tidak perlu menyimpan data transaksi secara permanen, nilai `hari` dilewatkan sebagai parameter saat kalkulasi biaya dijalankan.
3. Perhatikan kode dari class Pelanggan. Untuk apakah perintah `mobil.hitungBiayaMobil(hari)` dan `sopir.hitungBiayaSopir(hari)`?
 - Perintah tersebut digunakan untuk memanggil method kalkulasi biaya dari masing-masing objek Part (`Mobil` dan `Sopir`) dengan mengoper durasi sewa, di mana return value-nya akan dijumlahkan pada method `hitungBiayaTotal()` milik `Pelanggan`.
4. Perhatikan class `MainPercobaan2`. Untuk apakah sintaks `p.setMobil(m)` dan `p.setSopir(s)`?
 - Untuk menyuntikkan (**inject**) alamat referensi objek `m` (dari class `Mobil`) dan objek `s` (dari class `Sopir`) yang telah dibuat sebelumnya ke dalam atribut `mobil` dan `sopir` pada objek pelanggan `p`.
5. Untuk apakah proses `p.hitungBiayaTotal()`?
 - Untuk menghitung total biaya transaksi sewa pelanggan dengan menjumlahkan return value dari biaya sewa mobil dan biaya jasa sopir sesuai durasi hari.
6. Pada Langkah 7, `p.getMobil().getMerk()` memanggil dua method sekaligus secara berantai. Jelaskan urutan eksekusinya!

- Eksekusi berjalan berantai dari kiri ke kanan: method `p.getMobil()` dieksekusi lebih dulu untuk mengembalikan referensi objek `Mobil`, kemudian method `.getMerk()` dipanggil langsung pada objek `Mobil` tersebut untuk menghasilkan string merk mobil.
7. Andaikan `p.setMobil(m)` tidak pernah dipanggil lalu `p.hitungBiayaTotal()` dijalankan, error apa yang akan muncul? Jelaskan alasannya!
- Akan muncul runtime error `java.lang.NullPointerException` (`Cannot invoke "...hitungBiayaMobil(int)" because "this.mobil" is null`). Error terjadi karena atribut `this.mobil` masih bernilai default `null` (tidak menunjuk ke objek mana pun di heap memory), sehingga pemanggilan method pada referensi kosong tersebut ditolak oleh JVM.

2.3. Percobaan 3: Aggregation Multi-Role dan Penanganan NullPointerException (Kereta Api dan Pegawai)

Percobaan ketiga mengimplementasikan relasi di mana satu class Part (`Pegawai`) digunakan untuk mengisi dua atribut peran berbeda (`masinis` dan `asisten`) pada class `KeretaApi`. Pada saat objek `KeretaApi` diinisialisasi menggunakan constructor 3-parameter tanpa menyertakan asisten, pemanggilan `this.asisten.info()` menghasilkan `NullPointerException`. Masalah ini diatasi menggunakan **guard clause** (`if (this.asisten != null)`).

```
// KeretaApi.java (setelah perbaikan guard clause)
public String info() {
    String info = "";
    info += "Nama: " + this.nama + "\n";
    info += "Kelas: " + this.kelas + "\n";
    info += "Masinis: " + this.masinis.info() + "\n";
    if (this.asisten != null) {
        info += "Asisten: " + this.asisten.info() + "\n";
    }
    return info;
}
```

```
● rafi@santaro experiments % java ./MainPercobaan3.java
Nama: Gaya Baru
Kelas: Bisnis

Masinis:
NIP: 1234
Nama: Spongebob Squarepants
○ rafi@santaro experiments % |
```

2.3.1. Pertanyaan

1. Di dalam method `info()` pada class `KeretaApi`, baris `this.masinis.info()` dan `this.asisten.info()` digunakan untuk apa?
 - Digunakan untuk mendelegasikan pencetakan informasi data pegawai ke masing-masing objek `Pegawai` yang bertugas sebagai masinis dan asisten masinis.
2. Apa hasil output dari `MainPertanyaan` sebelum diperbaiki (Langkah 8)? Mengapa hal tersebut dapat terjadi?
 - Program berhenti mendadak dengan runtime error `NullPointerException`. Hal ini terjadi karena objek `KeretaApi` dibuat lewat constructor 3-parameter yang tidak mengisi atribut `asisten`, sehingga `this.asisten` bernilai `null` saat method `.info()` dipanggil.
3. Kaitkan dengan materi referensi objek: apa isi variabel `asisten` di dalam objek `KeretaApi` yang dibuat lewat constructor 3-parameter, sebelum guard clause ditambahkan?
 - Variabel `asisten` berisi nilai literal `null` (default value tipe referensi di Java). Artinya, variabel referensi tersebut sudah dialokasikan di dalam objek `KeretaApi`, namun belum menunjuk/merujuk ke alamat instance objek `Pegawai` mana pun di heap memory.
4. Setelah guard clause ditambahkan (Langkah 9), apakah objek `masinis` juga perlu dicek dengan cara yang sama? Perhatikan kedua constructor `KeretaApi`!
 - Berdasarkan kedua constructor `KeretaApi` (baik versi 3-parameter maupun 4-parameter), parameter `masinis` selalu diwajibkan sehingga secara logika alur program normalnya `masinis` tidak akan `null`. Namun, secara **defensive programming**, pengecekan `if (this.masinis != null)` tetap dianjurkan untuk mencegah crash jika pemanggil sengaja mengoper nilai `null`.
5. Kelas `Pegawai` dipakai lewat dua atribut berbeda (`masinis` dan `asisten`) pada `KeretaApi`. Apakah ini membuat `KeretaApi` punya dua objek `Pegawai` yang berbeda, atau satu objek yang dipakai dua kali?
 - Memiliki **dua objek `Pegawai` yang berbeda**. Pada method `main`, operator `new Pegawai(...)` dipanggil dua kali secara

terpisah (`masinis = new Pegawai(...)` dan `asisten = new Pegawai(...)`),
menghasilkan dua instance fisik objek independen di heap memory dengan
NIP dan Nama masing-masing.

2.4. Percobaan 4: Composition Array of Object dan Multiplicity (Gerbong, Kursi, Penumpang)

Percobaan keempat menggabungkan dua jenis relasi: relasi **Composition** antara `Gerbong` dan array `Kursi`, serta relasi **Aggregation** antara `Kursi` dan `Penumpang`. Seluruh objek `Kursi` diciptakan secara internal dan eksklusif di dalam method private `initKursi()` pada class `Gerbong`. Ditambahkan pula validasi agar kursi yang sudah terisi tidak dapat ditimpa oleh penumpang lain.

```
// Gerbong.java
private void initKursi() {
    for (int i = 0; i < arrayKursi.length; i++) {
        this.arrayKursi[i] = new Kursi(String.valueOf(i + 1));
    }
}

public void setPenumpang(Penumpang penumpang, int nomor) {
    if (this.arrayKursi[nomor - 1].getPenumpang() == null) {
        this.arrayKursi[nomor - 1].setPenumpang(penumpang);
    } else {
        System.out.println("Kursi nomor " + nomor + " sudah ditempati oleh "
            + this.arrayKursi[nomor - 1].getPenumpang().getNama() + "!");
    }
}
```

```
● rafi@santaro experiments % java ./MainPercobaan4.java
Kode: A
Nomor: 1
Penumpang: KTP: 12345
Nama: Krab

Nomor: 2
Nomor: 3
Nomor: 4
Nomor: 5
Nomor: 6
Nomor: 7
Nomor: 8
Nomor: 9
Nomor: 10

Kode: A
Nomor: 1
Penumpang: KTP: 12345
Nama: Krab

Nomor: 2
Nomor: 3
Nomor: 4
Nomor: 5
Nomor: 6
Nomor: 7
Nomor: 8
Nomor: 9
Nomor: 10

○ rafi@santaro experiments %
```

2.4.1. Pertanyaan

1. Pada main program dalam class MainPercobaan4, berapakah jumlah kursi dalam Gerbong A?
 - Terdapat **10 kursi** (sesuai argumen inisialisasi `new Gerbong("A", 10)`), di mana pada saat pengujian baru 1 kursi yang terisi oleh penumpang.
2. Perhatikan potongan kode if (`this.penumpang != null`) pada method `info()` dalam class `Kursi`. Apa maksud kode tersebut?
 - Potongan kode tersebut merupakan **guard clause** untuk memeriksa keterisian kursi. Jika kursi terisi (`penumpang != null`), method akan memanggil `penumpang.info()`. Jika kosong, proses pemanggilan diabaikan agar tidak menimbulkan `NullPointerException`.
3. Mengapa pada method `setPenumpang()` dalam class `Gerbong`, nilai nomor dikurangi dengan angka 1?
 - Karena penomoran kursi di dunia nyata berbasis 1 (**1-indexed**, kursi nomor 1 sampai 10), sedangkan indeks array pada bahasa Java berbasis 0 (**0-indexed**, indeks 0 sampai 9). Pengurangan `nomor - 1` diperlukan untuk memetakan nomor kursi ke indeks elemen array yang sesuai.
4. Instansiasi objek baru budi dengan tipe `Penumpang`, kemudian masukkan pada gerbong menimpa Mr. Krab (`gerbong.setPenumpang(budi, 1)`). Apakah yang terjadi? Apakah Java memberi error?
 - Pada kode asli sebelum validasi ditambahkan, Java **tidak memberi error ataupun peringatan**. Referensi objek penumpang lama (Mr. Krab) pada elemen kursi ke-1 langsung ditimpa (**overwritten**) dengan referensi objek `budi`.
5. Tuliskan potongan kode modifikasi sehingga tidak diperkenankan menduduki kursi yang sudah ada penumpang lain!
 - Modifikasi validasi pada method `Gerbong.setPenumpang()`:

```
public void setPenumpang(Penumpang penumpang, int nomor) {  
    if (this.arrayKursi[nomor - 1].getPenumpang() == null) {  
        this.arrayKursi[nomor - 1].setPenumpang(penumpang);  
    } else {  
        System.out.println("Peringatan: Kursi nomor " + nomor + " sudah  
ditempati oleh "
```

```
        + this.arrayKursi[nomor - 1].getPenumpang().getNama() + "!\n");  
    }  
}
```

6. Untuk kasus seperti apa kita memilih array of object, dan untuk kasus seperti apa kita memilih atribut bernama satu-satu?
- Array of object dipilih jika kelas Whole memiliki sekumpulan banyak objek Part yang sejenis (multiplicity 1..n), di mana tiap objek tidak memiliki peran semantik khusus yang berbeda (contoh: 10 kursi gerbong). Atribut bernama satu-satu dipilih jika jumlah objek Part sedikit/pasti dan tiap objek memiliki peran fungsional yang spesifik dan berbeda (contoh: `masinis` dan `asisten` pada `KeretaApi`).
7. Terapkan kriteria kode (siapa yang memanggil new) pada relasi Gerbong-Kursi dan Kursi-Penumpang: manakah Aggregation dan manakah Composition?
- `Gerbong` ke `Kursi` adalah **Composition**, karena objek `Kursi` diciptakan langsung secara internal di method `initKursi()` pada class `Gerbong` (`this.arrayKursi[i] = new Kursi(...)`). Sedangkan `Kursi` ke `Penumpang` adalah **Aggregation**, karena objek `Penumpang` dibuat mandiri di luar kelas (di `Main`) lalu disuntikkan via setter (`kursi.setPenumpang(penumpang)`).

2.5. Percobaan 5: Composition Kepemilikan Eksklusif (Mobil dan Mesin)

Percobaan kelima menguji implementasi Composition murni satu-ke-satu. Class `Mobil1` memiliki atribut `mesin` yang diinstansiasi secara internal di dalam constructor-nya menggunakan operator `new`. Class `Mobil1` sengaja tidak menyediakan method `setMesin()`, sehingga objek `Mesin` berstatus eksklusif dan siklus hidupnya terikat mutlak dengan objek `Mobil1`.

```
// Mobil1.java
public class Mobil1 {
    private String merek;
    private Mesin mesin;

    public Mobil1(String merek) {
        this.merek = merek;
        this.mesin = new Mesin(); // Komposisi murni: instansiasi internal
    }

    public void tampilkanInfo() {
        System.out.println("Mobil: " + this.merek);
        System.out.println("Mesin: " + this.mesin.getTipe());
    }
}
```

```
● rafi@santaro experiments % java ./MainPercobaan5.java
Mobil: Avanza
Mesin: 4-silinder
○ rafi@santaro experiments % |
```

2.5.1. Pertanyaan

1. Pada class Mobil, baris manakah yang menunjukkan bahwa Mesin adalah bagian yang “dimiliki secara eksklusif” oleh Mobil?
 - Ditunjukkan pada baris instansiasi di constructor `Mobil1`:
`this.mesin = new Mesin();` serta tidak tersedianya method setter `setMesin(Mesin mesin)`.
2. Apa yang terjadi secara desain jika ditambahkan method `setMesin(Mesin mesin)` pada class Mobil? Apakah relasi ini tetap Composition?
 - Relasi tersebut akan **kehilangan karakteristik Composition murninya** dan bergeser menjadi **Aggregation**. Adanya setter memungkinkan objek `Mesin` dari luar disuntikkan atau diganti sewaktu-waktu, merusak prinsip kepemilikan eksklusif dan keterikatan **lifecycle** mutlak.
3. Bandingkan dengan Percobaan 1 (Laptop-Processor): sebutkan satu perbedaan baris kode yang membuat salah satunya Aggregation dan yang lain Composition!
 - Pada Percobaan 1 (`Laptop`), objek part diterima lewat parameter constructor dari luar (`this.proc = proc;`), sedangkan pada Percobaan 5 (`Mobil1`), objek part dibuat langsung secara mandiri di dalam constructor (`this.mesin = new Mesin();`).
4. Jika objek mobil di-set null, apa yang terjadi pada objek Mesin miliknya? Bandingkan dengan nasib Processor pada Percobaan 1!
 - Objek `Mesin` akan otomatis kehilangan satu-satunya referensi yang mengarah kepadanya, menjadi **unreachable** di heap memory, dan segera dibersihkan oleh Garbage Collector. Sebaliknya pada Percobaan 1, objek `Processor` masih tetap hidup karena alamat referensinya sempat ditampung di variabel luar (`Processor p = ...`) di fungsi `main`.
5. Jika ditambahkan constructor kedua `public Mobil(String merek, Mesin mesin)`, apakah relasi berubah menjadi Aggregation? Jelaskan!
 - Ya, berubah menjadi **Aggregation**. Karena objek `Mesin` di-instansiasi di luar kelas `Mobil` terlebih dahulu kemudian disuntikkan lewat parameter constructor, membuktikan `Mesin` memiliki **lifecycle** independen.

2.6. Percobaan 6: Dependency / Uses-A (Laptop dan Printer)

Percobaan keenam mempelajari bentuk relasi paling longgar (**loose coupling**) yaitu **Dependency (uses-a)**. Class `Laptop1` tidak menyimpan objek `Printer` sebagai atribut instansiasi. Objek `Printer` hanya dilewatkan sesaat sebagai parameter method `cetakDokumen(Printer printer, String namaFile)`. Setelah method selesai dieksekusi, ikatan kedua objek langsung terlepas.

```
// Laptop1.java
public class Laptop1 {
    private String merk;

    public Laptop1(String merk) {
        this.merk = merk;
    }

    // Printer hanya masuk lewat parameter method, tidak disimpan sebagai atribut
    public void cetakDokumen(Printer printer, String namaFile) {
        System.out.println(merk + " mengirim dokumen ke printer...");
        printer.cetak(namaFile);
    }
}
```

```
● rafi@santaro experiments % java ./MainPercobaan6.java
Thinkpad mengirim dokumen ke printer...
[Epson L3110] Mencetak Laporan.pdf...
[Epson L3110] Selesai
○ rafi@santaro experiments % |
```

2.6.1. Pertanyaan

1. Apakah class Laptop pada percobaan ini memiliki atribut bertipe Printer? Bandingkan dengan Percobaan 1!
 - Tidak ada atribut bertipe `Printer` pada class `Laptop1`. Berbeda dengan Percobaan 1 di mana `Processor` disimpan secara permanen sebagai atribut instansiasi, pada Percobaan 6 objek `Printer` hanya diterima sebagai parameter method `cetakDokumen`.
2. Setelah method `cetakDokumen()` selesai dijalankan, apakah Laptop masih menyimpan referensi ke objek printer yang tadi dipakai? Jelaskan!
 - Tidak. Parameter `printer` hanya berstatus sebagai variabel lokal pada **stack frame** method `cetakDokumen()`. Begitu method selesai dieksekusi (**stack frame di-pop**), referensi tersebut hilang dan class `Laptop1` tidak menyimpan status apa pun mengenai objek printer.
3. Mengapa relasi Laptop-Printer pada percobaan ini disebut Dependency (uses-a), bukan Aggregation?
 - Karena tidak ada hubungan kepemilikan (**ownership / has-a**) maupun penyimpanan status (**state retention**). Class `Laptop1` hanya bergantung (**depends on**) sementara waktu kepada layanan class `Printer` untuk menyelesaikan operasi pencetakan dokumen.
4. Jika kode diubah sehingga Printer disimpan sebagai atribut class Laptop, apakah relasi berubah menjadi Aggregation? Jelaskan!
 - Ya, relasi tersebut berubah menjadi **Aggregation**. Karena objek `Printer` disimpan sebagai atribut instansiasi di dalam class `Laptop` dan referensinya dipertahankan sepanjang masa hidup objek `Laptop`.
5. Lengkapi tabel perbandingan relasi: Aggregation, Composition, dan Dependency!
 - Berikut adalah matriks perbandingan ketiga relasi kelas:

Relasi	Disimpan sbg Atribut?	Lokasi Pemanggilan <code>new</code>	Karakteristik Lifecycle
--------	-----------------------	--	-------------------------

Aggregation	Ya	Luar kelas (caller / <code>main</code>)	Part hidup mandiri (independent). Disuntikkan via constructor/setter.
Composition	Ya	Dalam kelas (<code>constructor</code>)	Part terikat mati (tight coupling). Part musnah bersama Whole. Tidak ada setter.
Dependency	Tidak	Bebas (caller / <code>main</code>)	Temporer (loose coupling). Objek hanya lewat sebagai parameter method.

3. Tugas

3.1. Tugas Mandiri 1: Studi Kasus Sistem RPG Battle Berorientasi Objek

Studi kasus mandiri yang dibangun adalah **Sistem Pertarungan RPG Battle** (`assignments`), yang mengintegrasikan 5 class domain (di luar `Main`):

- `Hero`: Superclass entitas karakter game.
- `Fighter`: Subclass `Hero` yang bertindak sebagai Whole.
- `Weapon`: Senjata khusus milik `Fighter` (**Composition**).
- `Item`: Aksesoris pertahanan/kekuatan yang dapat dipasang dan dilepas (**Aggregation**).
- `Potion`: Ramuan pemulih HP sekali pakai (**Dependency**).

```
// Fighter.java (mencakup Composition dan Aggregation)
public class Fighter extends Hero {
    // 1. Composition: Weapon dibuat secara internal dan eksklusif
    private Weapon weapon;
    // 2. Aggregation: Item dibuat mandiri di luar kelas dan disuntikkan
    private Item item;

    public Fighter(String name, int level, int health, int strength, int defense, int
speed,
                    String weaponName, int power, Item item) {
        super(name, level, health, strength, defense, speed);
        this.weapon = new Weapon(weaponName, power); // Composition
        this.item = item;                             // Aggregation
    }

    public void setItem(Item item) {
        this.item = item; // Setter untuk mengganti item
    }
}

// Hero.java (mencakup Dependency)
public void usePotion(Potion potion) {
    this.health += potion.getHealAmount();
    System.out.println(name + " menggunakan " + potion.getName() + "...");
}
```

Tinjauan Kriteria Tiga Relasi pada Studi Kasus:

1. **Composition** (`Fighter` → `Weapon`): Objek `Weapon` di-instantiasi langsung menggunakan operator `new` di dalam constructor `Fighter`. Class `Fighter` tidak menyediakan method `setWeapon()`, sehingga **lifecycle** `Weapon` terikat mati dengan `Fighter`.

2. **Aggregation (Fighter → Item):** Objek Item (Aegis Shield, Ring of Power) dibuat mandiri di luar kelas dan disuntikkan lewat parameter constructor atau setter `setItem()`. Saat item diganti, objek item lama tetap eksis di heap memory.
3. **Dependency (Hero / Fighter → Potion):** Objek Potion tidak disimpan sebagai atribut kelas, melainkan hanya dilewatkan sesaat via parameter method `usePotion(Potion potion)` dan segera lepas setelah nilai `health` bertambah.

```
● rafi@santaro assignments % java ./Main.java
=====
DEMONSTRASI RELASI KELAS (TUGAS MANDIRI JOBSHEET 4)
=====
[Aggregation] Objek Item dibuat independen: Aegis Shield
=== Status Fighter ===
Nama      : Aragorn (Level 10)
HP        : 100
Weapon    : Anduril Flame [Power: 50] -> (Composition)
Item      : Aegis Shield [Bonus STR: +5, DEF: +20] -> (Aggregation)

Aragorn menyerang dengan Anduril Flame (Total Damage: 80)!

[Dependency] Menggunakan Potion lewat parameter method usePotion():
Aragorn menggunakan Elixir of Life (+40 HP). Total HP: 140

[Aggregation] Ganti item ke: Ring of Power
Aragorn menyerang dengan Anduril Flame (Total Damage: 90)!
Objek shield lama masih eksis di memori: Aegis Shield
=====
○ rafi@santaro assignments % |
```

3.2. Tugas Mandiri 2: Pedoman Keputusan Desain Relasi Antar-Kelas

Dalam merancang arsitektur sistem berorientasi objek baru, keputusan memilih relasi ditentukan berdasarkan kebutuhan persistensi status (**state retention**) dan keterikatan siklus hidup (**lifecycle**) melalui pertanyaan kunci berikut:

1. **Apakah objek target perlu disimpan statusnya sepanjang masa hidup kelas pemanggil? Jika tidak** dan hanya dibutuhkan sesaat untuk menyelesaikan satu tugas operasi tertentu, gunakan relasi **Dependency (uses-a)**.
2. **Jika objek harus disimpan sebagai atribut (has-a), apakah objek part memiliki identitas mandiri dan masih bermakna jika objek whole dihapus?** Jika **ya** (seperti pelanggan pada pesanan, buku pada perpustakaan, atau aksesoris pada karakter), gunakan relasi **Aggregation (loose coupling via constructor/setter injection)**.
3. **Apakah objek part merupakan komponen internal eksklusif yang tidak dapat hidup atau tidak memiliki arti tanpa kehadiran whole? Jika ya** (seperti kursi pada gerbong, mesin pada mobil, atau baris detail nota), gunakan relasi **Composition (tight coupling dengan `new` internal tanpa setter)**.